

AI Itinerary Generator for Sri Lanka Tourism: A Heuristic Multi-Constraint Planner (Tour Ceylon)

Wanniarachchi P.M.R.

AI Itinerary Generator -- Tour Ceylon -- Unified Tourism Platform for Sri Lanka

Informatics Institute of Technology (IIT), affiliated with University of Westminster

BSc (Hons) Information Technology -- Software Engineering

Abstract

This paper reports the itinerary-planning component of Tour Ceylon, a unified modular monolith for Sri Lanka tourism. The component generates day-by-day trip schedules from curated government-style registries rather than from a learned ranker or a large language model. A six-step wizard collects moods (at most three), attraction selections, a USD budget split, and one hotel per destination, then a deterministic engine assigns destination blocks, clock times, and a PDF export. The catalogue comprises 350 attractions across 31 destinations (10 mood tags, 6 categories), 937 accommodations, 126 businesses, 417 travel agencies, and 2,906 tourist guides after registration filtering. Scheduling uses a mood-averaged start hour (default 08:30), inter-activity gaps of 2.0-3.0 h, a lunch snap in [12:00, 13:00], and an evening cap at 19:30. Location strings are normalised with an alias map plus SequenceMatcher at similarity 0.88. Validation is a 12-step HTTP smoke test of the full wizard-to-PDF path; no accuracy, precision, recall, or F1 scores are claimed because the planner is rule-based (the product name AI Itinerary Generator refers to automated scheduling, not a trained neural ranker). Limitations include the absence of road-network travel times and learned attraction ranking. The contribution is an explainable, deployable planner that shares identity with SOS and wellness through a single Flask API on port 5002.

Index Terms-- itinerary generation, heuristic scheduling, tourism information systems, constraint-based planning, Sri Lanka, Flask, PostgreSQL

I. INTRODUCTION

Foreign visitors to Sri Lanka typically assemble trips from fragmented lists of attractions, hotels, and licensed guides. Tour Ceylon's travel component addresses that gap with a browser wizard that ends in a timed day plan and a downloadable PDF. The module is the primary tourist-facing product of the platform (React 18.3 + Vite 5.4 on port 5180) and writes trip state into PostgreSQL tables consumed later by SOS hand-off and wellness sessions. This report documents methodology, measured catalogue sizes, validation, and limitations as implemented in the unified codebase--not as a hypothetical recommender.

A central design choice is explicit: although the module is named AI Itinerary Generator, the file `ai_service.py` does not call OpenAI. An `OPENAI_API_KEY` may be present in configuration but is unused. Generated itineraries are tagged `source: "database"`. The engine is therefore reproducible without a paid model API and is inspectable for a viva or IEEE methods section.

II. SYSTEM CONTEXT AND REQUIREMENTS

A. Role in Tour Ceylon

Tour Ceylon is a modular MVC monolith: one Flask 3.0.3 process (port 5002) serves Travel, SOS, and Wellness. JSON Web Tokens (Flask-JWT-Extended 4.6.0) identify the tourist. After planning, the same session can open the SOS PWA (port 5175) via a travel-to-SOS bridge. The itinerary component is therefore both a standalone planner and the identity gateway for safety and wellness modules.

B. Functional Scope

The wizard exposes six user steps: (1) attractions with 1-3 moods; (2) budget; (3) one hotel per destination; (4) generated itinerary; (5) discover (businesses, agencies, guides); (6) PDF export and history. Mood vocabulary is fixed at ten labels: Adventure, Authentic, Curious, Excited, Explore, Happy, Healing, Peaceful, Relaxed, and Spiritual. Attraction categories are Wild, Scenic, Pristine, Heritage, Essence, and Thrills.

III. METHODOLOGY

A. Technical Setup

Frontend: React 18.3, react-router-dom 6.26, Axios, Recharts (budget pie), Vite proxy of /api to 127.0.0.1:5002. Backend: Flask, SQLAlchemy 2.0, PostgreSQL database travel_app. PDF: WeasyPrint when available, else ReportLab 4.2.2. Hardware is a conventional development PC; no GPU is required.

TABLE I

Seeded catalogue sizes used at runtime (live PostgreSQL travel_app, August 2026).

Dataset	Source	Records used
Attractions	attractions.xlsx (sheet Attractions)	350
Accommodations	accomadation.xlsx	937
Business directory	business_directory.xlsx	126
Travel agencies	travel_agencies.xlsx	417
Tourist guides	tourist_guides.csv (3,634 raw)	2,906 after filters
Unique destinations (attractions)	Destination column	31 (28 after normalized_destination)
Mood tags / categories	Excel columns	10 / 6

B. Data Collection and Preprocessing

Registries live in backend/data/travel/. seed_data.py reads them with pandas 2.2.2 and openpyxl 3.1.5. Local-authority strings are mapped through an explicit alias table, then residual mismatches are aligned with Python difflib.SequenceMatcher at ratio threshold 0.88; flagged pairs are written to a review JSON. Hotel price text is parsed into price_min and price_max. Guide rows are dropped unless they have a valid registration and a known type {National, Chauffeur, Area, Site}. Attraction images are extracted from Excel embeddings. A leftover checklist in verify_seed.py still encodes 130 attractions / 944 hotels from an earlier catalogue; this report uses the live table counts (350 / 937), which match the current seed files.

C. Constraint Checks on Trip Input

Endpoint POST /api/trip-input requires days > 0, one to three moods, a non-empty attraction ID list, and

$$\text{unique_locations(selected attractions)} \leq \text{trip_days}$$

(1)

Equation (1) forbids visiting more distinct destinations than there are days. Moods stored on the trip are the union of attraction mood tags and UI moods, truncated to three.

D. Destination Block Allocation

Attractions are grouped by destination in user selection order (no travelling-salesman reordering). Let D be the number of destinations and N the trip length. Base length is $N // D$; remainder $r = N \bmod D$ is given to later destinations:

$$\text{block}_i = \text{floor}(N/D) + 1 [i \geq D - r]$$

(2)

Example encoded in comments: a 7-day trip Colombo -> Batticaloa -> Hambantota yields days 1-2, 3-4, and 5-7. If a destination occupies several days, attractions are spread one-per-day with overflow on the last day of the block. Spare days with no remaining sights receive a synthetic activity Explore {destination}.

E. Clock-Time Heuristic

Start hour is the mean of a mood lookup (decimal hours). Spiritual = 6.5, Adventure = 7.5, Explore = Excited = 8.0, Curious = 8.5, Happy = Authentic = 9.0, Healing = 9.5, Peaceful = 10.0, Relaxed = 10.5. Empty mood list defaults to 8.5 (08:30). The first activity may be delayed by a category/name bias, then clamped to [5.0, 14.0]. Gaps: 3.0 h for two activities, 2.5 h for three, 2.0 h for four or more. If the cursor lies in [12.0, 13.0) it snaps to 13.0 (lunch). If the cursor reaches 20.0 it is capped at 19.5.

F. Hotel Filter and Budget

Default split is 25% food, 35% accommodation, 15% shopping, 25% transport (must sum to 100%). Nightly USD budget is accommodation_amount / max(days - 1, 1), converted at 300 LKR per USD. Room rates: Single = price_min;

Family = price_max; Double = (price_min + price_max) / 2. Hotels whose nightly rate exceeds the cap are dropped. Matching is ILIKE on normalised local authority, authority, and address--not Haversine (unlike SOS).

G. Discover Recommendations

Businesses, agencies, and guides are filtered by destination substring. If empty, a hard-coded nearby-city map is tried; if still empty, a global top-100 fallback is returned. Remaining spend shown to the user is food + shopping + transport.

H. Approaches Considered and Final Choice

No sklearn models were trained on attractions. An LLM path was prototyped in naming only (unused API key). Collaborative filtering was not implemented because the catalogue has no large rating matrix. The final heuristic planner was selected for (i) explainability in an undergraduate viva, (ii) zero inference cost, and (iii) stable outputs for PDF export. scikit-learn 1.5.1 appears in requirements.txt solely for the wellness component.

IV. RESULTS

This component does not produce classifier accuracy. Reportable quantitative results are catalogue cardinalities (Table I), algorithm constants (Table II), and a 12-step integration test that asserts HTTP success from health check through PDF, not itinerary quality. There is no ablation of ranking models because ranking is binary mood inclusion, not a scored list (attractions are ordered alphabetically).

TABLE II

Implemented constants (from ai_service.py, trip_plan_service.py, budget_service.py).

Parameter	Value
Default day start	08:30 (decimal 8.5)
Mood start range	06:30-10:30
First-activity clamp	[05:00, 14:00]
Inter-activity gap (2 / 3 / 4+ stops)	3.0 h / 2.5 h / 2.0 h
Lunch window snap	[12:00, 13:00) -> 13:00
Evening cap	19:30
Location fuzzy threshold	0.88
LKR per USD	300
Default budget split (F/A/S/T)	25 / 35 / 15 / 25 %
Max moods	3
Hotels per destination	1
API smoke test steps	12

Persistence: trip drafts in user_trip_input and budget_split; generated plans as JSONB in generated_itinerary. History is listed by GET /api/itinerary/history.

V. DISCUSSION

A. What the Component Enables

A completed itinerary gives the tourist a timed plan and, more importantly for the platform, a logged-in identity that SOS and wellness reuse. Heuristic times make the PDF look like a real day rather than an unordered list, which matters for demonstration and for user trust even without an ML ranker.

B. Limitations

- No road-network or public-transport times; destination order is selection order, not TSP.
- Hotel matching is string ILIKE, so a beach hotel in a neighbouring Pradeshiya Sabha may be missed.
- Mood filtering is include/exclude; there is no relevance score.
- Activity durations and opening hours are not modelled; every attraction occupies one slot.
- A tight accommodation budget can return zero hotels (no relaxation strategy).
- Fuzzy match at 0.88 can mis-map local-authority names; review JSON exists but is manual.

- The 12-step test does not score whether 08:30 is a sensible start at a given site.

C. Future Work

With more time: OpenStreetMap driving matrices between consecutive stops; duration-aware packing; learning-to-rank from saved itineraries; automatic budget relaxation; A/B of mood-mean start versus user-chosen start. Those are explicitly out of the current release.

VI. CONCLUSION

The Tour Ceylon itinerary component is a constraint-based planner over 350 attractions and 937 hotels, with deterministic block allocation and clock heuristics. It is suitable to describe in IEEE methods as a rule-based information system, not as a machine-learning result. Its value in the overall system is a complete, exportable trip plus a shared tourist identity for SOS and wellness.

REFERENCES

- [1] P. J. Pedregosa et al. are not used in this component; scikit-learn in the monorepo supports wellness only.
- [2] Flask documentation, Pallets Projects, version 3.0, 2024. [Online]. Available: <https://flask.palletsprojects.com/>
- [3] SQLAlchemy 2.0 documentation, 2024. [Online]. Available: <https://docs.sqlalchemy.org/>
- [4] PostgreSQL Global Development Group, PostgreSQL 16 documentation.
- [5] React, "React 18 documentation," Meta, 2024.
- [6] Vite.js, "Vite 5 documentation," 2024.
- [7] Python Software Foundation, difflib.SequenceMatcher, Python 3 Library Reference.
- [8] ReportLab Ltd., ReportLab PDF Library 4.2.2 User Guide, 2024.
- [9] Sri Lanka Tourism Development Authority (SLTDA) style registries as ingested Excel/CSV catalogues (attractions, accommodation, agencies, guides) in backend/data/travel/.
- [10] Tour Ceylon, backend/app/services/ai_service.py and trip_plan_service.py, source code, 2026.
- [11] Tour Ceylon, backend/scripts/travel/test_api_flow.py, 12-step HTTP integration test, 2026.
- [12] Fielding, R. T., "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000 (REST constraints used by the /api surface).

Note: Do not report classifier F1 or millisecond inference for this component; those metrics belong to the wellness report.